Guía técnica de puesta en producción

Camping La Guineu · landing page + editor visual en Cloudflare

Para el desarrollador · Versión 1.0 · Octubre 2026 · Repositorio: **github.com/PiensaMcfly/laguineu-web**

1. Qué es y cómo funciona
2. Puesta en marcha (token, Cloudflare, secretos)
3. Conectar el dominio .com.ar, paso a paso
4. Seguridad y mantenimiento
5. Qué podría faltar (por prioridad)
6. Solución de problemas
7. Checklist de entrega al cliente
8. Estructura del repositorio

1. Qué es y cómo funciona

Landing page del camping con un **editor visual en** `/admin` para que el cliente cambie textos, fotos, colores, tipografías, secciones y videos de YouTube sin tocar código. Se publica en **Cloudflare Workers** (con archivos estáticos) en el plan gratuito, y cada cambio queda guardado en este repositorio con historial para volver atrás.

- **Qué se publica:** la carpeta `site/` (landing, editor, imágenes, scripts propios).
- **Servidor:** `functions/` contiene la lógica (login, publicar, versiones, imágenes) y `worker/index.js` la conecta con Cloudflare. `wrangler.jsonc` es la configuración.
- **Publicar:** el editor manda la página, los datos del editor y las imágenes nuevas (ya convertidas a WebP) al servidor; este hace **un solo commit** en GitHub y Cloudflare redespliega solo en 1 o 2 minutos.
- **Qué guarda el servidor:** solo `site/index.html`, `site/data/project.json` e imágenes en `site/Public/Images/uploads/`. Nunca puede modificar código.
- **Videos:** solo por link de YouTube (modo sin cookies). No se sube nada pesado al proyecto.
- **Secretos:** la contraseña, la clave de sesión y el token de GitHub viven solo en Cloudflare, nunca en el repo ni en el navegador del cliente.

2. Puesta en marcha

1. **Token de GitHub** (para que el editor pueda guardar). Entrá a `github.com/settings/personal-access-tokens/new`. Nombre *Editor La Guineu*; vencimiento 1 año (anotá la fecha); **Only select repositories** → solo `laguineu-web`; permiso **Contents: Read and write**. Generá y copialo: se muestra una sola vez.

   > No pegues el token en chats ni documentos: va directo a Cloudflare. Si se filtra, revocalo y creá otro.

2. **Cloudflare.** Cuenta gratis en `dash.cloudflare.com/sign-up` → **Workers & Pages** → **Create** → **Import a repository** → autorizá GitHub para `laguineu-web`. Nombre del proyecto: `laguineu-web` (debe coincidir con `name` en `wrangler.jsonc`). Build command vacío; deploy command por defecto (`npx wrangler deploy`). Cada push redespliega solo.

3. **Secretos** (Settings → Variables and secrets). Cargá estos tres con tipo **Secret**:

Nombre	Valor
`ADMIN_PASSWORD`	Contraseña del cliente: frase larga de **12 caracteres o más**. Entregársela por un canal privado.
`SESSION_SECRET`	Texto al azar de **32 caracteres o más** (comando abajo). Nunca un email ni una palabra reconocible.
`GITHUB_TOKEN`	El token del paso 1.

`GITHUB_REPO` y `GITHUB_BRANCH` ya están en `wrangler.jsonc`: **no los cargues a mano en el panel** (cada despliegue los pisa con lo que dice ese archivo). Si cargás una clave como "Variable" en vez de "Secret", se ve en el panel y el próximo despliegue puede borrarla.

```
$b = New-Object byte[] 36; [System.Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($b);
[Convert]::ToBase64String($b)
```

4. **Probar.** `https://laguineu-web.TU-SUBDOMINIO.workers.dev/` muestra la landing; `/admin/` pide la contraseña. Hacé un cambio chico, tocá **Publicar** y verificá que en 1 o 2 minutos se vea. Si el login dice "no configurado", el mensaje indica **qué variable falta o es corta** (nunca muestra los valores).
5. **Opcional pero recomendado:** bloqueo por intentos fallidos con KV. Creá un KV (Workers & Pages → KV), copiá su ID y agregá en `wrangler.jsonc`: `"kv_namespaces": [{ "binding": "RATE", "id": "EL_ID" }]`. No lo asocies solo desde el panel (el próximo despliegue lo borraría). Resultado: 5 contraseñas incorrectas seguidas desde la misma IP bloquean el login 15 minutos.

3. Conectar el dominio .com.ar, paso a paso

Los nombres exactos de los botones pueden variar levemente en Cloudflare y NIC; la secuencia es la misma. **Recomendación:** que el dominio quede **a nombre del cliente** (es su marca) y vos controles solo el DNS desde tu cuenta de Cloudflare.

A. Registrar el dominio en NIC Argentina

1. El cliente entra a `nic.ar` con su **CUIT/CUIL y Clave Fiscal** y registra el dominio (.com.ar). El pago es con tarjeta. Costo anual: confirmarlo en NIC (una nota de 2026 indica unos $8.500, pero el precio cambia).
2. Anotar la **fecha de vencimiento**: si el dominio vence, el sitio y el editor dejan de funcionar. Poner recordatorios 30 y 7 días antes.

B. Agregar el dominio a Cloudflare

3. En Cloudflare: **Add a domain** (Onboard a domain) → escribir `tudominio.com.ar` → plan **Free** → continuar (no hace falta importar registros DNS).
4. Cloudflare muestra **dos nameservers** (algo como `xxx.ns.cloudflare.com` y `yyy.ns.cloudflare.com`). Copiarlos.

C. Delegar el dominio en NIC

5. En `nic.ar` → **Mis dominios** → elegir el dominio → **Delegar / Delegaciones** → agregar una delegación nueva con los **dos nameservers de Cloudflare** y confirmar. (NIC no administra los registros DNS: solo permite delegarlos.)
6. Esperar la propagación (de minutos hasta **24 o 48 horas**). En Cloudflare, tocar **Check nameservers** hasta que el dominio figure como **Active**.

D. Apuntar el dominio al sitio

7. En el proyecto `laguineu-web` → **Settings** → **Domains & routes** → **Add** → **Custom domain** → `tudominio.com.ar`. Repetir con `www.tudominio.com.ar`. Cloudflare crea los registros y el certificado **HTTPS automático** (puede demorar unos minutos).
8. Redirigir **www → sin www** (o al revés): Rules → Redirect Rules → crear una regla simple. Así hay una sola dirección oficial.
9. Verificar: candado de HTTPS, la landing carga, `/admin/` funciona y se puede **publicar un cambio de prueba** desde el dominio nuevo.

E. Cerrar los pendientes de seguridad

10. **Cloudflare Access** (gratis hasta 50 usuarios): Zero Trust → Access → Applications → **Self-hosted** → dominio con las rutas `/admin` y `/api` → política **Allow** solo para los emails del cliente y el tuyo (entran con un código por mail). Nadie ve ni el login.
11. **Apagar la dirección .workers.dev**: en `wrangler.jsonc` poner `"workers_dev": false`, subir el cambio y verificar. Importante: Access protege el dominio propio pero **no la dirección .workers.dev**, por eso hay que apagarla.
12. Opcional: regla de límite de pedidos (Security → WAF → Rate limiting rules) sobre `/api/login`.

F. Después de salir al aire

- **Google Business Profile:** poner el dominio nuevo como sitio web de la ficha del camping.
- **Google Search Console:** agregar el dominio (verificación por registro TXT en el DNS de Cloudflare) y enviar el sitemap cuando exista.
- **Redes sociales:** actualizar el link en Instagram y Facebook.
- **Mail:** si el cliente quiere `info@tudominio.com.ar` sin casillas propias, Cloudflare Email Routing (gratis) reenvía a un Gmail.

4. Seguridad y mantenimiento

Tarea	Cómo / cuándo
Cambiar la contraseña del cliente	Editar el secreto `ADMIN_PASSWORD` en Cloudflare. Se aplica al instante.
Cerrar todas las sesiones	Cambiar el secreto `SESSION_SECRET`.
Renovar el token de GitHub	Antes del vencimiento (1 año): crear uno nuevo con los mismos permisos y reemplazar el secreto `GITHUB_TOKEN`. Si vence, "Publicar" falla con un mensaje de permisos.
Renovar el dominio	Anualmente en NIC.
Backups	El repositorio de GitHub es el respaldo (con historial). Opcional: un `git clone` mensual en tu compu.
Notas de publicación	Quedan en el historial público del repo. Avisar al cliente que no escriba datos sensibles (hay un cartel en la ventana de publicar).

- **Protecciones activas en el repo:** no se puede borrar la rama principal ni reescribir su historial; GitHub bloquea pushes con claves; alertas de dependencias; análisis automático de código (CodeQL).
- **En el sitio:** política de seguridad de contenido (solo scripts propios), cookie `HttpOnly` / `Secure` / `SameSite=Strict`, verificación de origen en cada pedido, filtro del servidor que rechaza scripts, formularios o marcos no permitidos, e integridad (SRI) de las librerías de terceros del editor.
- No hay base de datos, por lo que no existe inyección SQL.
- **Tu cuenta de GitHub:** mantener la verificación en dos pasos y activar "Keep my email addresses private" y "Block command line pushes that expose my email". Cuando termines de configurar, cerrar sesión de la herramienta de línea de comandos con `gh auth logout`.

5. Qué podría faltar (por prioridad)

#	Mejora	Por qué conviene	Prioridad
1	Datos para compartir y Google (descripción, imagen para WhatsApp/Facebook, favicon, datos del negocio, sitemap)	Hoy la página solo tiene título. Al compartir el link por WhatsApp se ve sin foto ni descripción, y Google muestra menos información. Es lo más rentable para un camping.	ALTA
2	Aliviar el peso de las fotos originales	Las imágenes de `site/Public/Images` pesan unos 8 MB (varias en .jfif). Convertirlas a WebP puede bajarlo a menos de 2 MB: mejora la carga en celulares con datos móviles. Las que sube el cliente ya se convierten solas.	ALTA
3	Cloudflare Access + límite de intentos (KV/WAF)	Esconde el login y frena fuerza bruta. Pasos en las secciones 2 y 3.	ALTA
4	Recordatorios de vencimientos	Dominio (anual), token de GitHub (1 año). Si vencen, el sitio o la publicación dejan de funcionar.	ALTA
5	Compilar Tailwind	Hoy el navegador calcula los estilos (unos 400 KB de JavaScript). Compilado queda en un CSS de ~30 KB: carga más rápida y mejor puntaje en Google.	MEDIA
6	Estadísticas de visitas (Cloudflare Web Analytics)	Gratis y sin cookies. Hay que permitir ese script en la política de seguridad y en el filtro del servidor.	MEDIA
7	Monitoreo de disponibilidad (UptimeRobot u otro)	Aviso por mail si el sitio se cae.	MEDIA
8	Pruebas en celulares y navegadores reales	Revisar Android e iPhone, Chrome y Safari antes del lanzamiento.	MEDIA
9	Botón de llamada y datos de contacto revisados	Enlace `tel:` para llamar con un toque; confirmar teléfono, horarios y dirección.	MEDIA
10	Textos alternativos en todas las fotos	Accesibilidad y SEO. El editor tiene el campo "Descripción de la imagen".	BAJA
11	Aviso de privacidad / cookies	El mapa de Google y los videos de YouTube pueden usar cookies. Un aviso simple alcanza.	BAJA
12	Acuerdo de mantenimiento con el cliente	Definir quién paga la renovación del dominio, qué incluye el soporte y cuánto tarda en responder.	BAJA
13	Entorno de pruebas	Para probar cambios grandes sin tocar el sitio real (hoy las vistas previas están apagadas).	BAJA

6. Solución de problemas

Qué ves	Causa probable y solución
"El servidor todavía no está configurado" + una lista	Falta o es corta una variable. El mensaje nombra cuál (`ADMIN_PASSWORD` mínimo 12 caracteres, `SESSION_SECRET` mínimo 32). Cargarla como **Secret** y volver a desplegar.
"Contraseña incorrecta"	Revisar el secreto `ADMIN_PASSWORD` (espacios al final, mayúsculas). Con KV activo, 5 fallos seguidos bloquean 15 minutos.
"El servidor no tiene permiso en GitHub"	Token vencido, revocado o sin permiso **Contents: Read and write** sobre `laguineu-web`. Crear uno nuevo y reemplazar `GITHUB_TOKEN`.
"No encuentro el repositorio en GitHub"	`GITHUB_REPO` mal escrito en `wrangler.jsonc`, o el token no incluye ese repo.
"Se guardó otro cambio al mismo tiempo" (409)	Dos publicaciones simultáneas. Reintentar.
"La página tiene un script/elemento no permitido"	El filtro del servidor rechazó contenido (scripts, formularios, marcos de sitios que no sean Google Maps o YouTube). Sacar ese elemento.
Publicó pero no se ve el cambio	Esperar 1 o 2 minutos y recargar con Ctrl+F5. Ver el último despliegue en Cloudflare (Deployments).
El despliegue falla en Cloudflare	Ver el log de la compilación. Revisar que `name` en `wrangler.jsonc` sea `laguineu-web` y que exista la carpeta `site/`.
Las claves desaparecen después de un despliegue	Estaban cargadas como "Variable" y no como "Secret". Volver a cargarlas como Secret.
El editor muestra una versión vieja	Usar **Descartar cambios** (vuelve a lo último publicado). El editor guarda borradores locales que se descartan solos si alguien publicó desde otro lado.
El login pide Access y el cliente no entra	Agregar su email a la política **Allow** de la aplicación de Access.

7. Checklist de entrega al cliente

☐ Dominio con HTTPS funcionando y **.workers.dev** apagado.
☐ Contraseña definitiva (frase larga) entregada por un canal privado.
☐ Cloudflare Access con los emails correctos.
☐ Prueba completa desde el dominio: ingresar, editar, publicar, abrir Versiones y volver atrás.
☐ Guía del cliente entregada (**Guia-del-editor-para-el-cliente.pdf**).
☐ Recordatorios cargados: vencimiento del token (1 año) y del dominio.
☐ Tokens de prueba revocados; verificación en dos pasos activa en GitHub; `gh auth logout` hecho.
☐ Datos de contacto, horarios y tarifas de la página revisados con el cliente.
☐ Acordado quién paga la renovación del dominio y cómo se pide soporte.

8. Estructura del repositorio

```
laguineu-web/
├ site/                 lo que se publica
|  ├ index.html         landing (la reescribe el editor al publicar)
|  ├ admin/             editor visual (index.html + app.js)
|  ├ Src/               estilos y scripts propios (site.js, tw-config.js, laguineu.css)
|  ├ vendor/            Tailwind guardado en una versión fija
|  ├ Public/Images/     fotos (las nuevas van a uploads/)
|  ├ data/project.json  datos del editor (se crea al publicar)
|  ├ _headers           cabeceras de seguridad
```

```
│  └ 404.html, robots.txt
├ functions/            lógica del servidor (login, publicar, versiones, imágenes)
├ worker/index.js       enrutador del servidor para Cloudflare Workers
├ wrangler.jsonc        configuración de Cloudflare
└ docs/                 estas guías en PDF
```